

Họ & tên: Dương Thiện Quý

Mssv: 3123410298

BÀI THỰC HÀNH SỐ 5

Bài 1:

a. Tiến trình cha chuyển đổi số đầu tiên (`argv[1]`) là một số nguyên lớn hơn 3 cho tiến trình

con thông qua message queue. Tiến trình con nhận, tính giá trị $n! = 1 * 2 * \dots * n$ và ghi nó vào

message queue. Tiến trình cha nhận và xuất dữ liệu ra màn hình. Sử dụng message queue để

giải quyết vấn đề.

>./baitap1A.out 4

4! = 24

b. Hãy thực hiện 2 tiến trình hoàn toàn riêng biệt, không sử dụng tiến trình cha con. Giải quyết lại vấn đề trên.

Code:

- Server:

```
bai1_server.c
OS_HomeWork > BT_Tuan3_Part2 > C bai1_server.c > ...
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <sys/ipc.h>
4 #include <sys/msg.h>
5
6 // Định nghĩa cấu trúc thông điệp (Khớp với Client)
7 struct msgbuf {
8     long mtype;
9     int n;
10    long long result;
11 };
12
13 int main() {
14     // 1. Tạo hoặc lấy ID của Message Queue
15     key_t key = ftok(".", 1);
16     if (key == -1) {
17         perror("Lỗi ftok");
18         return 1;
19     }
20
21     int msgid = msgget(key, 0666 | IPC_CREAT);
22     if (msgid == -1) {
23         perror("Lỗi msgget");
24         return 1;
25     }
26
27     printf("Server (Bai 1) đang chạy. Cho yêu cầu...\n");
28
29     struct msgbuf msg;
30     // Chạy vòng lặp liên tục
31     while (1) {
32         // 2. Chờ nhận thông điệp từ Client
33         // Lọc lấy các thông điệp có mtype = 1
34         if (msgrcv(msgid, &msg, sizeof(struct msgbuf) - sizeof(long), 1, 0) == -1) {
35             perror("Lỗi msgrcv");
36             continue;
37         }
38     }
39 }
```

```
bai1_server.c X
OS_HomeWork > BT_Tuan3_Part2 > C bai1_server.c > ...
13 int main() {
20
29 struct msgbuf msg;
30 // Chạy ngầm liên tục
31 while (1) {
32     // 2. Chờ nhận thông điệp từ Client
33     // Lọc lấy các thông điệp có mtype = 1
34     if (msgrcv(msgid, &msg, sizeof(struct msgbuf) - sizeof(long), 1, 0) == -1) {
35         perror("Loi msgrcv");
36         continue; // Bỏ qua nếu lỗi, chờ tin nhắn tiếp theo
37     }
38
39     printf("Server: Nhan duoc n = %d. Dang tinh toan...\n", msg.n);
40
41     // 3. Tính toán giá trị giai thừa (Sử dụng long long để tránh tràn số)
42     long long fact = 1;
43     for (int i = 1; i <= msg.n; i++) {
44         fact *= i;
45     }
46
47     // 4. Đóng gói kết quả và gửi lại cho Client
48     msg.mtype = 2; // Loại 2 dành cho chiều Server -> Client
49     msg.result = fact;
50
51     if (msgsnd(msgid, &msg, sizeof(struct msgbuf) - sizeof(long), 0) == -1) {
52         perror("Loi msgsnd");
53     } else {
54         printf("Server: Da gui ket qua %lld tro lai Client.\n", fact);
55     }
56 }
57
58 return 0;
59 }
60
```

- Client:

```
bai1_client.c X
OS_HomeWork > BT_Tuan3_Part2 > C bai1_client.c > ...
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <sys/ipc.h>
4 #include <sys/msg.h>
5
6 // Định nghĩa cấu trúc thông điệp
7 struct msgbuf {
8     long mtype; // Loại thông điệp (Bắt buộc phải là số dương)
9     int n; // Dữ liệu: số nguyên n cần tính giai thừa
10     long long result; // Dữ liệu: kết quả n!
11 };
12
13 int main(int argc, char *argv[]) {
14     // Kiểm tra số lượng đối số
15     if (argc != 2) {
16         printf("Su dung: %s <so_nguyen_n>\n", argv[0]);
17         return 1;
18     }
19
20     int n = atoi(argv[1]);
21     if (n <= 3) {
22         printf("Loi: So nguyen phai lon hon 3.\n");
23         return 1;
24     }
25
26     // 1. Tạo hoặc lấy ID của Message Queue dùng chung
27     key_t key = ftok(".", 1);
28     if (key == -1) {
29         perror("Loi ftok");
30         return 1;
31     }
32
33     int msgid = msgget(key, 0666 | IPC_CREAT);
34     if (msgid == -1) {
35         perror("Loi msgget");
36     }
37 }
```

```
bai1_client.c x
OS_HomeWork > BT_Tuan3_Part2 > C bai1_client.c > ...
13 int main(int argc, char *argv[]) {
28     if (key == -1) {
32
33         int msgid = msgget(key, 0666 | IPC_CREAT);
34         if (msgid == -1) {
35             perror("Loi msgget");
36             return 1;
37         }
38
39         // 2. Chuẩn bị thông điệp gửi đi
40         struct msgbuf msg;
41         msg.mtype = 1; // Loại 1 dành cho chiều Client -> Server
42         msg.n = n;
43         msg.result = 0;
44
45         // Gửi thông điệp
46         if (msgsnd(msgid, &msg, sizeof(struct msgbuf) - sizeof(long), 0) == -1) {
47             perror("Loi msgsnd");
48             return 1;
49         }
50         printf("Client: Da gui n = %d. Dang cho ket qua...\n", n);
51
52         // 3. Chờ nhận phản hồi từ Server
53         // Lọc lấy các thông điệp có mtype = 2 (Server trả về)
54         if (msgrcv(msgid, &msg, sizeof(struct msgbuf) - sizeof(long), 2, 0) == -1) {
55             perror("Loi msgrcv");
56             return 1;
57         }
58
59         // In kết quả theo đúng định dạng yêu cầu
60         printf("%d! = %lld\n", msg.n, msg.result);
61
62         return 0;
63     }
}
```

Kết quả:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
quy@quy-VirtualBox:~/BTTH$ cd OS_HomeWork/
quy@quy-VirtualBox:~/BTTH/OS_HomeWork$ cd BT_Tuan3_Part2/
quy@quy-VirtualBox:~/BTTH/OS_HomeWork/BT_Tuan3_Part2$ ./bail_serv
er.out
Server (Bai 1) đang chạy. Cho yêu cầu...
Server: Nhận được n = 10. Đang tính toán...
Server: Đã gửi kết quả 3628800 trở lại Client.
[]

bash - BT_Tuan3_Part2 + v [] [] ... | [] x
quy@quy-VirtualBox:~/BTTH/OS_HomeWork/BT_Tuan3_Part2$ ./bail_client.out 10
Client: Đã gửi n = 10. Đang chờ kết quả...
quy@quy-VirtualBox:~/BTTH/OS_HomeWork/BT_Tuan3_Part2$ []
```

Bài 2:

Tiến trình cha đọc hai số nguyên và một thao tác +, -, x, / và chuyển tất cả cho tiến trình con. Quá trình con tính toán kết quả và trả về cho tiến trình cha. Quá trình cha xuất kết quả

```
>./baitap2A.out 4 6 +
```

4 + 6 = 10

Hãy giải quyết vấn đề trên với kỹ thuật message queue.

Hãy tách 2 tiến trình trên thành 2 tiến trình hoàn toàn độc lập. Giải quyết lại vấn đề trên.

Code:

- Server:

```
bai2_server.c x
OS_HomeWork > BT_Tuan3_Part2 > C bai2_server.c > ...
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <sys/ipc.h>
4  #include <sys/msg.h>
5
6  // Định nghĩa cấu trúc thông điệp (Khớp với Client)
7  struct msgbuf {
8      long mtype;
9      int a;
10     int b;
11     char op;
12     double result;
13     int error;
14 };
15
16 int main() {
17     // Tạo / liên kết Message Queue
18     key_t key = ftok(".", 1);
19     if (key == -1) {
20         perror("Lỗi ftok");
21         return 1;
22     }
23
24     int msgid = msgget(key, 0666 | IPC_CREAT);
25     if (msgid == -1) {
26         perror("Lỗi msgget");
27         return 1;
28     }
29
30     printf("Server May Tinh (Bai 2) đang chay...\n");
31
32     struct msgbuf msg;
33     // Chạy ngằm chờ đợi tin nhắn
34     while (1) {
35         // Nhận các tin nhắn từ Client
36         if (msgreceive(msgid, &msg, 0) == -1) {
37             perror("Lỗi nhận tin nhắn");
38             continue;
39         }
40
41         printf("Server: Nhan yeu cau tinh %d %c %d\n", msg.a, msg.op, msg.b);
42         msg.error = 0; // Khởi tạo cờ không lỗi
43
44         // Bóc tách và xử lý toán học
45         switch (msg.op) {
46             case '+': msg.result = msg.a + msg.b; break;
47             case '-': msg.result = msg.a - msg.b; break;
48             case 'x': msg.result = msg.a * msg.b; break;
49             case '*': msg.result = msg.a * msg.b; break;
50             case '/':
51                 if (msg.b == 0) {
52                     msg.error = 1; // Lỗi chia cho 0
53                 } else {
54                     msg.result = (double)msg.a / msg.b;
55                 }
56                 break;
57             default:
58                 msg.error = 1; // Toán tử không hợp lệ
59         }
60
61         // Đóng gói kết quả và gửi phản hồi (mtype = 2)
62         msg.mtype = 2;
63         if (msgsnd(msgid, &msg, sizeof(struct msgbuf) - sizeof(long), 0) == -1) {
64             perror("Lỗi msgsnd");
65         } else {
66             printf("Server: Da gui ket qua tra lai Client.\n");
67         }
68     }
69
70     return 0;
71 }
```

- Client:

```
bai2_client.c x
OS_HomeWork > BT_Tuan3_Part2 > C bai2_client.c > ...
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <sys/ipc.h>
5  #include <sys/msg.h>
6
7  // Định nghĩa cấu trúc thông điệp chứa các toán hạng và toán tử
8  struct msgbuf {
9      long mtype;
10     int a;
11     int b;
12     char op; // Phép toán: '+', '-', 'x', '/'
13     double result;
14     int error; // Cờ báo lỗi (VD: chia cho 0)
15 };
16
17 int main(int argc, char *argv[]) {
18     if (argc != 4) {
19         printf("Su dung: %s <a> <b> <phép_toán>\n", argv[0]);
20         printf("Vi du: %s 4 6 +\n", argv[0]);
21         return 1;
22     }
23
24     int a = atoi(argv[1]);
25     int b = atoi(argv[2]);
26     char op = argv[3][0];
27
28     // Tạo hoặc liên kết Message Queue
29     key_t key = ftok(".", 1);
30     if (key == -1) {
31         perror("Lỗi ftok");
32         return 1;
33     }
34
35     int msgid = msgget(key, 0666 | IPC_CREAT);
36     if (msgid == -1) {
37         perror("Lỗi msgget");
38         return 1;
39     }
40
41     // Đóng gói dữ liệu gửi đi
42     struct msgbuf msg;
43     msg.mtype = 1; // mtype = 1 (Gửi đi Server)
44     msg.a = a;
45     msg.b = b;
46     msg.op = op;
47     msg.error = 0;
48
49     if (msgsnd(msgid, &msg, sizeof(struct msgbuf) - sizeof(long), 0) == -1) {
50         perror("Lỗi msgsnd");
51         return 1;
52     }
53     printf("Client: Da gui yeu cau %d %c %d. Dang cho...\n", a, op, b);
54
55     // Chờ nhận kết quả từ Server (mtype = 2)
56     if (msgrcv(msgid, &msg, sizeof(struct msgbuf) - sizeof(long), 2, 0) == -1) {
57         perror("Lỗi msgrcv");
58         return 1;
59     }
60
61     // Xử lý và in kết quả
62     if (msg.error) {
63         printf("Client: Server bao loi (Phep chia cho 0 hoac toan tu khong hop le).\n");
64     } else {
65         printf("Ket qua: %d %c %d = %.2f\n", msg.a, msg.op, msg.b, msg.result);
66     }
67
68     return 0;
69 }
```

Kết quả:



```
quy@quy-VirtualBox:~/BTTH/OS_HomeWork/BT_Tuan3_Pa
rt2$ ./bai2_server.out
Server May Tinh (Bai 2) dang chay...
Server: Nhan yeu cau tinh 10 1 0
Server: Da gui ket qua tra lai Client.
Server: Nhan yeu cau tinh 10 x 100
Server: Da gui ket qua tra lai Client.

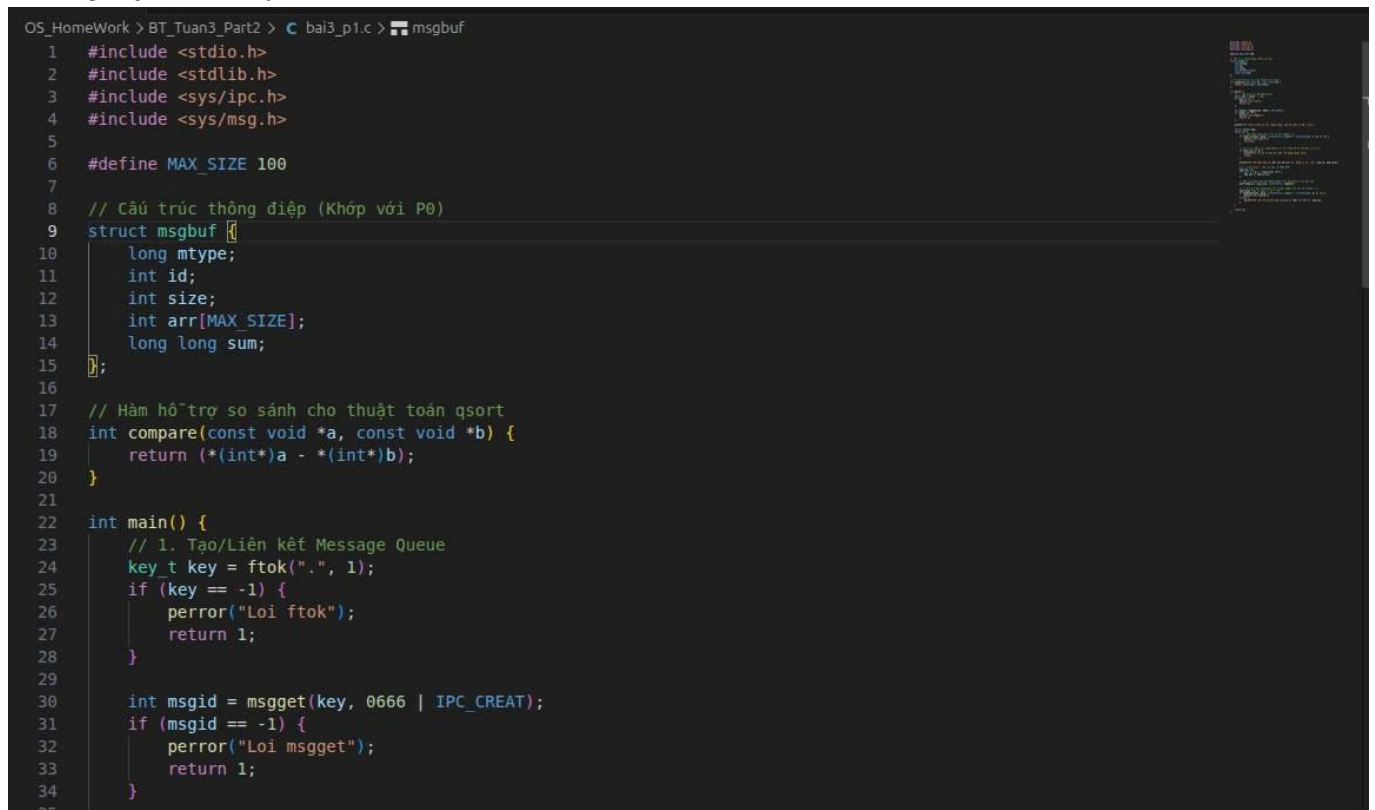
quy@quy-VirtualBox:~/BTTH/OS_HomeWork/BT_Tuan3_Pa
rt2$ ./bai2_client.out 10 100 x
Client: Da gui yeu cau 10 x 100. Dang cho...
Ket qua: 10 x 100 = 1000.00
quy@quy-VirtualBox:~/BTTH/OS_HomeWork/BT_Tuan3_Pa
rt2$
```

Bài 3:

Tạo ra 2 tiến trình P0 và P1. Tiến trình P0 đọc từ file nhiều dãy số nguyên liên tiếp (mỗi dãy có thể có số phần tử khác nhau). Sau đó tiến trình P0 lần lượt gửi các dãy sang P1. P1 thực hiện sắp xếp mỗi dãy theo thứ tự tăng dần, đồng thời tính tổng của dãy đó, sau đó gửi dãy kết quả và tổng tương ứng về P0. P0 sẽ ghi lại kết quả vào file sao cho: dãy có tổng nhỏ nhất sẽ được ghi đầu tiên, sau đó các dãy có tổng lớn hơn lần lượt ghi tiếp theo. Hãy thực hiện bài toán với message queue.

Code:

- Tiến trình P1:



```
OS_HomeWork > BT_Tuan3_Part2 > c bai3_p1.c > msgbuf
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <sys/ipc.h>
4 #include <sys/msg.h>
5
6 #define MAX_SIZE 100
7
8 // Cấu trúc thông điệp (Khớp với P0)
9 struct msgbuf {
10     long mtype;
11     int id;
12     int size;
13     int arr[MAX_SIZE];
14     long long sum;
15 };
16
17 // Hàm hỗ trợ so sánh cho thuật toán qsort
18 int compare(const void *a, const void *b) {
19     return (*(int*)a - *(int*)b);
20 }
21
22 int main() {
23     // 1. Tạo/Liên kết Message Queue
24     key_t key = ftok(".", 1);
25     if (key == -1) {
26         perror("Loi ftok");
27         return 1;
28     }
29
30     int msgid = msgget(key, 0666 | IPC_CREAT);
31     if (msgid == -1) {
32         perror("Loi msgget");
33         return 1;
34     }
35 }
```

OS_HomeWork > BT_Tuan3_Part2 > C bai3_p1.c > msgbuf

```
22 int main() {
31     if (msgid == -1) {
35
36         printf("P1 (Tien trinh xu ly): Dang chay, cho du lieu tu P0...\n");
37
38         struct msgbuf msg;
39         while (1) {
40             // 2. Chờ nhận mảng cần xử lý từ P0 (mtype = 1)
41             if (msgrcv(msgid, &msg, sizeof(struct msgbuf) - sizeof(long), 1, 0) == -1) {
42                 perror("Loi msgrcv");
43                 continue;
44             }
45
46             // Nếu kích thước gửi sang bằng 0 => Tín hiệu P0 báo đã đọc hết file
47             if (msg.size == 0) {
48                 printf("P1: Da xu ly het du lieu. P1 dang thoat.\n");
49                 break;
50             }
51
52             printf("P1: Da nhan day so [%d] gom %d phan tu. Dang xu ly...\n", msg.id, msg.size);
53
54             // 3. Tính tổng và sắp xếp dãy số nhận được
55             msg.sum = 0;
56             for (int i = 0; i < msg.size; i++) {
57                 msg.sum += msg.arr[i];
58             }
59
60             // Sắp xếp mảng tăng dần bằng thuật toán QuickSort tích hợp sẵn
61             qsort(msg.arr, msg.size, sizeof(int), compare);
62
63             // 4. Gửi kết quả (mảng đã xếp + tổng) ngược lại cho P0 (mtype = 2)
64             msg.mtype = 2; // Loại 2: P1 -> P0
65             if (msgsnd(msgid, &msg, sizeof(struct msgbuf) - sizeof(long), 0) == -1) {
66                 perror("Loi msgsnd");
67             }
68         }
69     }
70 }
71
72 return 0;
73 }
```

OS_HomeWork > BT_Tuan3_Part2 > C bai3_p1.c > msgbuf

```
22 int main() {
39     while (1) {
41         if (msgrcv(msgid, &msg, sizeof(struct msgbuf) - sizeof(long), 1, 0) == -1) {
42             perror("Loi msgrcv");
43             continue;
44         }
45
46         // Nếu kích thước gửi sang bằng 0 => Tín hiệu P0 báo đã đọc hết file
47         if (msg.size == 0) {
48             printf("P1: Da xu ly het du lieu. P1 dang thoat.\n");
49             break;
50         }
51
52         printf("P1: Da nhan day so [%d] gom %d phan tu. Dang xu ly...\n", msg.id, msg.size);
53
54         // 3. Tính tổng và sắp xếp dãy số nhận được
55         msg.sum = 0;
56         for (int i = 0; i < msg.size; i++) {
57             msg.sum += msg.arr[i];
58         }
59
60         // Sắp xếp mảng tăng dần bằng thuật toán QuickSort tích hợp sẵn
61         qsort(msg.arr, msg.size, sizeof(int), compare);
62
63         // 4. Gửi kết quả (mảng đã xếp + tổng) ngược lại cho P0 (mtype = 2)
64         msg.mtype = 2; // Loại 2: P1 -> P0
65         if (msgsnd(msgid, &msg, sizeof(struct msgbuf) - sizeof(long), 0) == -1) {
66             perror("Loi msgsnd");
67         } else {
68             printf("P1: Da tra ve ket qua cho day so [%d] toi P0.\n", msg.id);
69         }
70     }
71
72     return 0;
73 }
```

- Tiến trình P0:

OS_HomeWork > BT_Tuan3_Part2 > C bai3_p0.c > main()

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <sys/ipc.h>
4  #include <sys/msg.h>
5  #include <string.h>
6
7  #define MAX_SIZE 100
8
9  // Cấu trúc thông điệp truyền qua Queue
10 struct msgbuf {
11     long mtype;
12     int id;           // ID/Dòng của dãy số để phân biệt
13     int size;         // Số lượng phần tử
14     int arr[MAX_SIZE]; // Dãy số nguyên
15     long long sum;    // Tổng để trả về
16 };
17
18 // Cấu trúc lưu trữ dữ liệu để P0 sắp xếp ở cuối
19 struct ArrayData {
20     int id;
21     int size;
22     int arr[MAX_SIZE];
23     long long sum;
24 };
25
26 int main() {
27     // 1. Tạo Message Queue
28     key_t key = ftok(".", 1);
29     if (key == -1) {
30         perror("Lỗi ftok");
31         return 1;
32     }
33
34     int msgid = msgget(key, 0666 | IPC_CREAT);
35     if (msgid == -1) {
```

OS_HomeWork > BT_Tuan3_Part2 > C bai3_p0.c > main()

```
26 int main() {
34     int msgid = msgget(key, 0666 | IPC_CREAT);
35     if (msgid == -1) {
36         perror("Lỗi msgget");
37         return 1;
38     }
39
40     // Tăng giới hạn queue để tránh deadlock
41     struct msqid_ds buf;
42     msgctl(msgid, IPC_STAT, &buf);
43     buf.msg_qbytes = 65536; // Tăng từ 16384 lên 65536 bytes
44     msgctl(msgid, IPC_SET, &buf);
45
46     // 2. Mở file "data" để đọc
47     FILE *f_in = fopen("data", "r");
48     if (!f_in) {
49         perror("Lỗi mở file 'data'. Vui lòng tạo file 'data' trước");
50         return 1;
51     }
52
53     struct msgbuf msg;
54     char line[1024];
55     int line_count = 0;
56
57     printf("P0: Bắt đầu đọc dữ liệu từ file 'data'...\n");
58     fflush(stdout);
59     // Đọc từng dòng dãy số
60     while (fgets(line, sizeof(line), f_in)) {
61         msg.mtype = 1; // Loại 1: Gửi cho P1
62         msg.id = line_count;
63         msg.size = 0;
64
65         // Tách các số trên dòng
66         char *token = strtok(line, " \t\n");
```

OS_HomeWork > BT_Tuan3_Part2 > C bai3_p0.c > main()

```
26 int main() {
66 while (fgets(line, sizeof(line), f_in)) {
67     char *token = strtok(line, " \t\n");
68     while (token != NULL && msg.size < MAX_SIZE) {
69         msg.arr[msg.size++] = atoi(token);
70         token = strtok(NULL, " \t\n");
71     }
72     // Gửi dãy số
73     if (msg.size > 0) {
74         if (msgsnd(msgid, &msg, sizeof(struct msgbuf) - sizeof(long), 0) == -1) {
75             perror("Loi msgsnd");
76         } else {
77             printf("P0: Da gui day so [%d] (%dphan tu) cho P1.\n", msg.id, msg.size);
78             line_count++;
79         }
80     }
81 }
82 fclose(f_in);
83
84 // Gửi thông điệp size=0 để báo cho P1 biết P0 đã đọc xong file
85 msg.mtype = 1;
86 msg.size = 0;
87 msgsnd(msgid, &msg, sizeof(struct msgbuf) - sizeof(long), 0);
88
89 // 3. Chờ nhận lại dữ liệu đã xử lý từ P1 (mtype = 2)
90 struct ArrayData *results = malloc(line_count * sizeof(struct ArrayData));
91 int received_count = 0;
92
93 printf("P0: Dang cho P1 tra ve ket qua...\n");
94 while (received_count < line_count) {
95     if (msgrcv(msgid, &msg, sizeof(struct msgbuf) - sizeof(long), 2, 0) == -1) {
96         perror("Loi msgrcv");
97         continue;
98     }
99 }
```

OS_HomeWork > BT_Tuan3_Part2 > C bai3_p0.c > main()

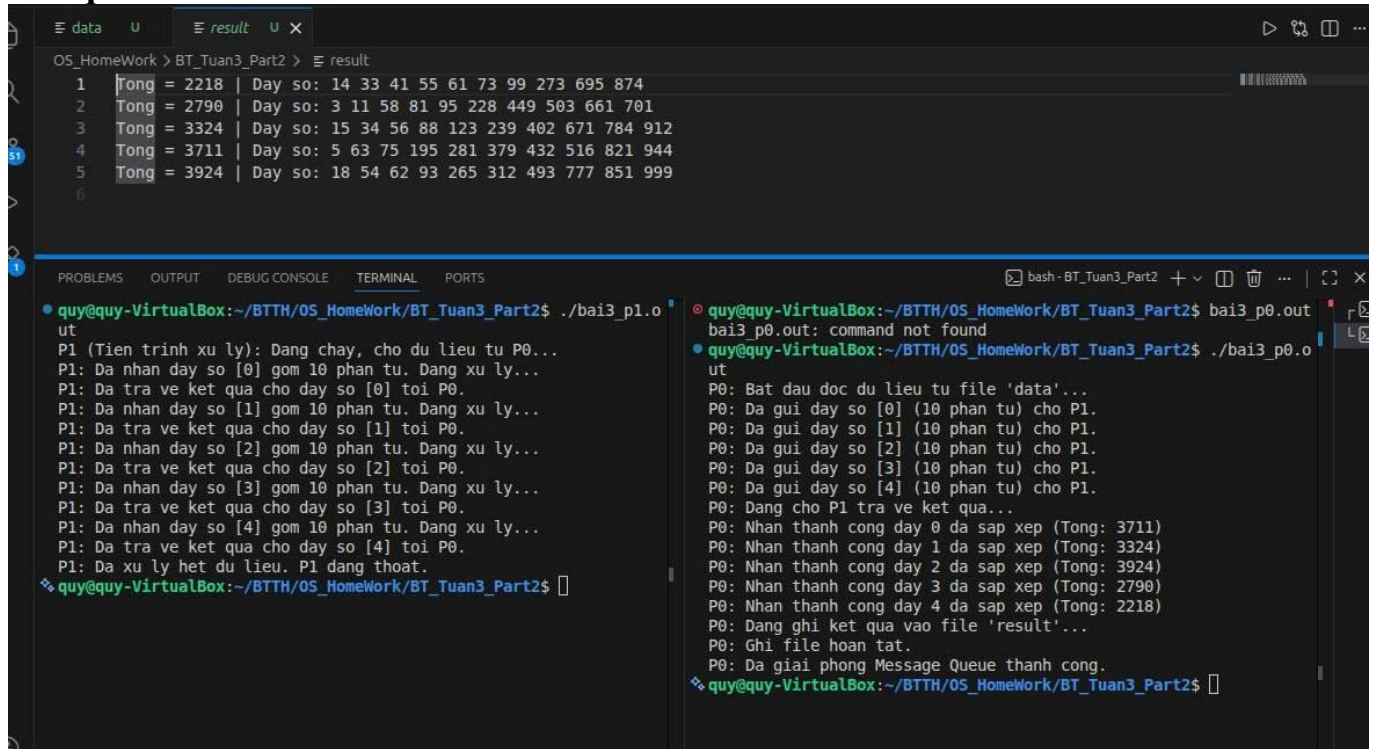
```
26 int main() {
94 while (received_count < line_count) {
95     if (msgrcv(msgid, &msg, sizeof(struct msgbuf) - sizeof(long), 2, 0) == -1) {
99
100         results[received_count].id = msg.id;
101         results[received_count].size = msg.size;
102         for (int i = 0; i < msg.size; i++) {
103             results[received_count].arr[i] = msg.arr[i];
104         }
105         results[received_count].sum = msg.sum;
106
107         printf("P0: Nhan thanh cong day %d da sap xep (Tong: %lld)\n", msg.id, msg.sum);
108         received_count++;
109     }
110
111     // 4. Sắp xếp toàn bộ các dãy dựa trên tổng của chúng (Tăng dần)
112     for (int i = 0; i < line_count - 1; i++) {
113         for (int j = i + 1; j < line_count; j++) {
114             if (results[i].sum > results[j].sum) {
115                 struct ArrayData temp = results[i];
116                 results[i] = results[j];
117                 results[j] = temp;
118             }
119         }
120     }
121
122     // 5. Ghi các dãy đã được sắp xếp ra file "result"
123     FILE *f_out = fopen("result", "w");
124     if (!f_out) {
125         perror("Loi mo file result");
126     } else {
127         printf("P0: Dang ghi ket qua vao file 'result'...\n");
128         for (int i = 0; i < line_count; i++) {
129             fprintf(f_out, "Tong = %lld | Day so: ", results[i].sum);
130         }
131     }
132 }
```

```
OS_HomeWork > BT_Tuan3_Part2 > C bai3_p0.d You can paste the image from the clipboard.
26  int main() {
112  for (int i = 0; i < line_count; i++) {
121
122  // 5. Ghi các dãy đã được sắp xếp ra file "result"
123  FILE *f_out = fopen("result", "w");
124  if (!f_out) {
125      perror("Loi mo file result");
126  } else {
127      printf("P0: Dang ghi ket qua vao file 'result'...\n");
128      for (int i = 0; i < line_count; i++) {
129          fprintf(f_out, "Tong = %lld | Day so: ", results[i].sum);
130          for (int j = 0; j < results[i].size; j++) {
131              fprintf(f_out, "%d ", results[i].arr[j]);
132          }
133          fprintf(f_out, "\n");
134      }
135      fclose(f_out);
136      printf("P0: Ghi file hoan tat.\n");
137  }
138
139  free(results);
140
141  // 6. Huỷ bỏ System V Message Queue
142  if (msgctl(msgid, IPC_RMID, NULL) == -1) {
143      perror("Loi msgctl IPC_RMID");
144  } else {
145      printf("P0: Da giai phong Message Queue thanh cong.\n");
146  }
147
148  return 0;
149 }
150
```

File data:

```
data x result u
OS_HomeWork > BT_Tuan3_Part2 > data
1 432 821 195 944 516 63 281 75 5 379
2 88 123 671 34 912 239 56 784 402 15
3 999 54 312 777 62 851 18 493 93 265
4 661 3 81 449 95 228 58 701 11 503
5 33 874 14 695 55 99 273 73 61 41
6
```

Kết quả:



The image shows a VS Code editor with a C program in `bai3_p1.o` and its execution output in the terminal. The program processes an array of 10 numbers, calculating their sum and printing the result for each day (0 to 4).

```
OS_HomeWork > BT_Tuan3_Part2 > result
1 | Tong = 2218 | Day so: 14 33 41 55 61 73 99 273 695 874
2 | Tong = 2790 | Day so: 3 11 58 81 95 228 449 503 661 701
3 | Tong = 3324 | Day so: 15 34 56 88 123 239 402 671 784 912
4 | Tong = 3711 | Day so: 5 63 75 195 281 379 432 516 821 944
5 | Tong = 3924 | Day so: 18 54 62 93 265 312 493 777 851 999
6
```

```
quy@quy-VirtualBox:~/BTTH/OS_HomeWork/BT_Tuan3_Part2$ ./bai3_p1.o
P1 (Tien trinh xu ly): Dang chay, cho du lieu tu P0...
P1: Da nhan day so [0] gom 10 phan tu. Dang xu ly...
P1: Da tra ve ket qua cho day so [0] toi P0.
P1: Da nhan day so [1] gom 10 phan tu. Dang xu ly...
P1: Da tra ve ket qua cho day so [1] toi P0.
P1: Da nhan day so [2] gom 10 phan tu. Dang xu ly...
P1: Da tra ve ket qua cho day so [2] toi P0.
P1: Da nhan day so [3] gom 10 phan tu. Dang xu ly...
P1: Da tra ve ket qua cho day so [3] toi P0.
P1: Da nhan day so [4] gom 10 phan tu. Dang xu ly...
P1: Da tra ve ket qua cho day so [4] toi P0.
P1: Da xu ly het du lieu. P1 dang thoat.
quy@quy-VirtualBox:~/BTTH/OS_HomeWork/BT_Tuan3_Part2$
```

```
quy@quy-VirtualBox:~/BTTH/OS_HomeWork/BT_Tuan3_Part2$ bai3_p0.out
bai3_p0.out: command not found
quy@quy-VirtualBox:~/BTTH/OS_HomeWork/BT_Tuan3_Part2$ ./bai3_p0.o
P0: Bat dau doc du lieu tu file 'data'...
P0: Da gui day so [0] (10 phan tu) cho P1.
P0: Da gui day so [1] (10 phan tu) cho P1.
P0: Da gui day so [2] (10 phan tu) cho P1.
P0: Da gui day so [3] (10 phan tu) cho P1.
P0: Da gui day so [4] (10 phan tu) cho P1.
P0: Dang cho P1 tra ve ket qua...
P0: Nhan thanh cong day 0 da sap xep (Tong: 3711)
P0: Nhan thanh cong day 1 da sap xep (Tong: 3324)
P0: Nhan thanh cong day 2 da sap xep (Tong: 3924)
P0: Nhan thanh cong day 3 da sap xep (Tong: 2790)
P0: Nhan thanh cong day 4 da sap xep (Tong: 2218)
P0: Dang ghi ket qua vao file 'result'...
P0: Ghi file hoan tat.
P0: Da giai phong Message Queue thanh cong.
quy@quy-VirtualBox:~/BTTH/OS_HomeWork/BT_Tuan3_Part2$
```